



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Specifica Tecnica_G

Versione	0.0.8
Stato	In corso
Redazione	Giulia Barzon, Joseph Grant, Chiara Grossele, Matteo Cuo- go
Verifica_G	Verificato
Approvazione	Da approvare
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica
0.0.8	18/04/2026	Joseph Grant	Aggiunti i hook frontend	Elisa Marchioro
0.0.7	18/04/2026	Matteo Cuogo	Aggiunta pattern Registry	Giulia Barzon, Elisa Marchioro
0.0.6	17/04/2026	Matteo Cuogo	Aggiunta sezione Design Pattern	Giulia Barzon, Elisa Marchioro
0.0.5	17/04/2026	Chiara Grossele	Aggiunta sezione Tracciamento dei requisiti	Matteo Cuogo, Elisa Marchioro
0.0.4	17/04/2026	Joseph Grant	Aggiunta descrizione delle componenti frontend	Chiara Grossele, Matteo Cuogo
0.0.3	15/04/2026	Chiara Grossele	Modifica struttura e aggiunte alla sezione Architettura backend	Joseph Grant, Matteo Cuogo, Giulia Barzon
0.0.2	14/04/2026	Joseph Grant	Aggiunta tecnologie e architettura	Chiara Grossele, Giulia Barzon
0.0.1	28/03/2026	Giulia Barzon	Creazione del documento, scrittura introduzione	Tommaso Tom- bacco

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Approccio alla redazione	1
1.3	Scopo del prodotto	1
1.4	Glossario _G	1
1.5	Riferimenti	1
1.5.1	Riferimenti informativi	2
2	Tecnologie	3
2.1	Framework frontend	3
2.2	Librerie frontend	3
2.3	Linguaggio frontend	4
2.4	Tecnologie di build	4
2.5	Tecnologie di test	4
2.6	Framework api _G backend	4
2.7	Linguaggio backend	5
2.8	Server backend	5
2.9	Tecnologie deployment	5
3	Architettura del backend	6
3.1	Presentation layer	6
3.2	Application logic	6
3.3	Business logic	6
3.3.1	Gestione dei prompt _G (ai_strategies.py)	7
3.3.2	Orchestrazione delle operazioni (operation_registry.py)	7
3.3.3	Integrazione degli LLM (model_strategies.py)	7
3.3.4	Servizi di supporto al dominio (text_cleaning.py)	7
3.4	Rappresentazione UML per l'architettura del backend	7
4	Deployment	8
4.1	Strategia di Containerizzazione	8
4.2	Backend (Flask _G)	8
4.3	Frontend (React _G + Nginx _G)	8
4.4	Orchestrazione: Docker Compose _G	8
5	Architettura del frontend	9
5.1	Model	9
5.2	View	9
5.3	ViewModel	9
5.4	Topbar	10
5.5	Sidebar	11
5.6	EditorButton	12
5.7	SaveDocumentModal	13
5.8	MarkDownEditor	13
5.9	HistoryList	14
5.10	HistoryCard	15
5.11	UseEditor	16
5.12	Rappresentazione UML per il frontend	16

6	Design Pattern	17
6.1	Strategy pattern	17
6.1.1	Problema	17
6.1.2	Applicazione: promptG strategies (ai_strategies.py)	17
6.1.3	Applicazione: model strategies (model_strategies.py)	18
6.1.4	Caching (get_model)	18
6.2	Registry pattern	19
6.2.1	Problema	19
6.2.2	Applicazione: operation_registry.py	19
7	Tracciamento dei requisiti	20
7.1	Notazione dei requisiti	20
7.2	Tracciamento dei requisiti funzionali	20
7.3	Tracciamento dei requisiti di qualità	24
7.4	Tracciamento dei requisiti di vincolo	24

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la **Specifica Tecnica_G** del prodotto *Second Brain*, proposto dall'azienda *Zucchetti* al gruppo ByteMe.

Il documento descrive in dettaglio:

- le **tecnologie** adottate (linguaggi, framework, librerie, strumenti);
- l'**architettura** logica e fisica del sistema, con i pattern architetturali scelti;
- i **design pattern** applicati a livello di componenti, con motivazione delle scelte;
- la **documentazione API_G** tra frontend e backend;
- i **requisiti soddisfatti** con riferimento al documento *Analisi dei Requisiti_G* v1.4.1.

Il documento è rivolto al team di sviluppo come guida implementativa e a committente e proponente come verifica_G della coerenza tra scelte progettuali e requisiti concordati.

1.2 Approccio alla redazione

Il documento viene redatto in modo **incrementale**, assicurando coerenza con gli sviluppi in corso. Non può essere considerato completo fino al rilascio della versione 1.0.0.

1.3 Scopo del prodotto

Il progetto *Second Brain* mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall'integrazione di Large Language Model (LLM). L'applicazione web consentirà all'utente di:

- scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- interagire con un LLM per riassumere, correggere, riscrivere e tradurre testi;
- ottenere analisi critiche multi-prospettiche secondo il metodo dei *Sei Cappelli per Pensare* di Edward De Bono;
- delegare all'AI la stesura completa di un testo tramite Distant Writing_G.

1.4 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

1.5 Riferimenti

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0
- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

1.5.1 Riferimenti informativi

- *Analisi dei Requisiti_G* versione 2.0.0 (Ultimo accesso: 01/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- *Piano di Qualifica* versione 1.0.0 (Ultimo accesso: 01/04/2026)
PB - Piano di Qualifica v2.0.0
- Verbali interni ed esterni
- Glossario_G v2.0.0 (Ultimo accesso: 01/04/2026)
Documenti interni - Glossario_G v2.0.0
- Repository_G ufficiale di EasyMDE_G: <https://github.com/Ionaru/easy-markdown-editor>
(Ultimo accesso: 24/03/2026)
- Documentazione linguaggio markdown_G - CommonMark: <https://commonmark.org/>
(Ultimo accesso: 20/02/2026)
- Documentazione React_G 19: <https://react.dev>
Ultimo accesso: 27/03/2026
- Principi del Bulletproof React_G (repository_G ufficiale): <https://github.com/alan2207/bulletproof-react>
(Ultimo accesso: 27/03/2026)
- Documentazione Flask_G: <https://flask.palletsprojects.com>
(Ultimo accesso: 10/03/2026)
- Documentazione Docker_G: <https://docs.docker.com>
(Ultimo accesso: 27/02/2026)
- Documentazione TypeScript: <https://www.typescriptlang.org/docs/>
(Ultimo accesso: 27/03/2026)
- Slide del corso di Ingegneria del Software
(Ultimo accesso: 27/03/2026)
- Principi SOLID (slide prof. Cardin)
- Pattern architetturali (slide prof. Cardin)

2 Tecnologie

2.1 Framework frontend

Nome	versione	Descrizione
React _G	19.2.0	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Tabella 2: Descrizione del framework frontend

2.2 Librerie frontend

Nome	versione	Descrizione
EasyMDE _G	2.20.0	Editor Markdown _G con anteprima in tempo reale, basato su CodeMirror.
Axios	1.7.9	Client HTTP basato su Promise per effettuare richieste REST dal browser e da Node.js.
TanStack Query	-	Libreria per la gestione del fetching, caching e sincronizzazione dello stato server in React _G .
CSS Modules _G	-	Sistema di scope locale per i fogli di stile CSS, che evita conflitti tra classi in componenti diversi.
React _G Hot Toast	2.6.0	Libreria leggera per la visualizzazione di notifiche toast personalizzabili in React _G .
eslint	9.31.1	Strumento di linting per JavaScript e TypeScript, che aiuta a mantenere uno stile di codice coerente e a individuare errori.
Zustand	5.0.12	Libreria di state management leggera, utilizzata per la gestione dello stato globale e la persistenza dei dati nel LocalStorage.
Lucide-React _G	1.8.0	Libreria di icone vettoriali utilizzata per la UI dell'applicazione.

Tabella 3: Descrizione delle librerie frontend

2.3 Linguaggio frontend

Nome	versione	Descrizione
TypeScript	-	Superset tipizzato di JavaScript che aggiunge tipizzazione statica e migliora la manutenibilità del codice.
TSX	-	Estensione di TypeScript che consente di scrivere markup JSX _G all'interno di file tipizzati, usata per i componenti React _G .

Tabella 4: Descrizione dei linguaggi frontend

2.4 Tecnologie di build

Nome	versione	Descrizione
Vite _G	7.3.1	Build tool moderno e veloce per applicazioni web, con supporto nativo a ES modules e Hot Module Replacement.

Tabella 5: Descrizione delle tecnologie di build

2.5 Tecnologie di test

Nome	versione	Descrizione
Vitest	4.1.4	Framework di testing per applicazioni Vite _G , compatibile con l'API _G di Jest e ottimizzato per TypeScript.
Pytest	7.0.0	Framework di testing per Python, con sintassi semplice e supporto a fixture, parametrizzazione e plugin.
testing library React _G	16.3.2	Libreria per il testing di componenti React _G , che fornisce utilities per interagire con i componenti in modo simile a come un utente reale lo farebbe.

Tabella 6: Descrizione delle tecnologie di test

2.6 Framework api_G backend

Nome	versione	Descrizione
Flask _G	-	Microframework Python per lo sviluppo di API _G REST, leggero ed estensibile tramite plugin.

Tabella 7: Descrizione del framework api_G backend

2.7 Linguaggio backend

Nome	versione	Descrizione
Python	3.12-slim	Linguaggio di programmazione ad alto livello, utilizzato per la logica di business e l'implementazione delle API _G .

Tabella 8: Descrizione del linguaggio backend

2.8 Server backend

Nome	versione	Descrizione
-	-	-

Tabella 9: Descrizione del server backend

2.9 Tecnologie deployment

Nome	versione	Descrizione
Docker _G	-	Piattaforma per la containerizzazione delle applicazioni, che garantisce ambienti riproducibili e portabili.

Tabella 10: Descrizione delle tecnologie di deployment

3 Architettura del backend

Il sistema "Second Brain" adotta il pattern architetturale **Layered Architecture**. L'adozione di questa struttura garantisce una netta separazione delle responsabilità e riduce l'accoppiamento tra i moduli software. Il flusso di esecuzione è strettamente unidirezionale (top-down): ogni strato funge da astrazione per quello superiore, gestendo esclusivamente la logica di propria competenza.

Nel sistema gli strati sono progettati come "strati chiusi": una richiesta transita solo verso lo strato immediatamente sottostante, garantendo che i dettagli implementativi di un livello rimangano isolati rispetto agli altri. Coerentemente con i requisiti di progetto, il backend è progettato per essere **stateless**, delegando la persistenza dei dati interamente al client. L'architettura logica si articola nei seguenti livelli:

- **Presentation layer:** Gestione dell'interfaccia utente (UI/UX) e inoltro delle richieste tramite `APIG REST`;
- **Application logic:** Orchestrazione del flusso applicativo e instradamento delle richieste;
- **Business logic:** Core funzionale del sistema e gestione dell'integrazione con i modelli LLM.

3.1 Presentation layer

Rappresenta il livello più esterno dell'architettura, fisicamente disaccoppiato dal backend e implementato nell'applicativo frontend. Ha il compito di gestire l'esperienza utente, raccogliere gli input testuali e le configurazioni scelte, inoltrando il tutto al backend tramite chiamate asincrone. Questo strato è privo di logica di dominio o di elaborazione semantica, delegando interamente il carico computazionale al server.

3.2 Application logic

Costituisce il punto di ingresso dell'applicazione lato server. Nel progetto, questo livello è implementato nel modulo di routing del framework `FlaskG` (`app.py`).

Questo strato ricopre le seguenti responsabilità:

- Agisce come **REST Controller** e coordinatore del flusso di esecuzione.
- Riceve le richieste `HTTP POST` e valida sintatticamente i payload `JSON` in ingresso.
- Interroga il registro delle operazioni per recuperare la configurazione necessaria, traducendo la richiesta del client in chiamate verso la Business Logic.
- Gestisce le eccezioni di sistema, garantendo che il client riceva sempre risposte standardizzate e sicure in caso di errore.

L'Application logic non possiede conoscenza delle regole di dominio, della struttura specifica dei `promptG` o dei protocolli di rete utilizzati per contattare i provider AI.

3.3 Business logic

È il nucleo del sistema, dove risiedono le regole funzionali. Questo strato è isolato dal framework web (`FlaskG`) e dal protocollo `HTTP`, garantendo l'indipendenza della logica di business dall'infrastruttura di comunicazione. Utilizza il pattern comportamentale **Strategy** per gestire la variabilità degli algoritmi di generazione e dei modelli in modo estensibile.

I componenti principali si dividono in servizi di dominio e di integrazione:

- **Gestione dei Prompt_G**: Definizione delle istruzioni per l'IA;
- **Orchestrazione delle Operazioni**: Mappatura dinamica tra richieste e strategie;
- **Integrazione Modelli LLM**: Astrazione dei client per i provider AI;
- **Servizi di Supporto**: Post-elaborazione dell'output.

3.3.1 Gestione dei prompt_G (ai_strategies.py)

Incapsula la logica di costruzione dei contesti. Tramite il pattern Strategy, definisce le regole per le operazioni editoriali (SimplePromptStrategy), le traduzioni (TranslationStrategy) e l'analisi critica basata sul metodo dei Sei Cappelli di De Bono (DeBonoHatStrategy).

3.3.2 Orchestrazione delle operazioni (operation_registry.py)

Implementa il pattern **Registry**. Contiene le strutture dati (OperationConfig) che associano dinamicamente gli identificatori delle operazioni alle corrette strategie di prompt_G e ai modelli corrispondenti. Questo componente funge da "collante" tra i livelli, fornendo all'Application Layer oggetti preconfigurati e pronti all'uso.

3.3.3 Integrazione degli LLM (model_strategies.py)

Astrae le complessità di rete per la comunicazione con i provider esterni (infrastruttura Zucchetti o API_G compatibili OpenAI). Gestisce l'iniezione delle variabili d'ambiente (API_G key e base URL) e l'invocazione dei modelli. Grazie all'interfaccia AIModelStrategy, espone modelli come **Llama 3.2_G** e **Gemma 3_G** in modo polimorfico.

3.3.4 Servizi di supporto al dominio (text_cleaning.py)

Sottomodulo funzionale che implementa algoritmi di post-processing basati su espressioni regolari (regex) per "pulire" le risposte dei modelli da metatesto o preamboli indesiderati.

3.4 Rappresentazione UML per l'architettura del backend

4 Deployment

Il sistema *Second Brain* utilizza la tecnologia di containerizzazione **Docker_G** per garantire l'isolamento dei componenti, la portabilità tra diversi ambienti di sviluppo e produzione e la gestione semplificata delle dipendenze.

4.1 Strategia di Containerizzazione

L'architettura è suddivisa in due micro-servizi orchestrati tramite **docker_G-compose**, che permette di definire reti virtuali interne e volumi per la persistenza o il *hot-reloading* durante lo sviluppo.

4.2 Backend (Flask_G)

Il container del backend è ottimizzato per l'esecuzione di script Python in un ambiente leggero e sicuro.

- **Immagine di base:** `python:3.12-slim`. La scelta della variante *slim* riduce l'ingombro dell'immagine, contenendo solo le librerie essenziali per il runtime Python 3.12.
- **Configurazione:** La directory di lavoro è impostata su `/app`. Il sistema installa le dipendenze tramite `pip` senza utilizzare la cache per mantenere l'immagine snella.
- **Networking:** Il container espone la porta 8000.
- **Variabili d'ambiente:** Il backend riceve dinamicamente tramite il file `.env` le chiavi necessarie per il funzionamento degli LLM.

4.3 Frontend (React_G + Nginx_G)

Per il frontend è stata adottata una strategia di **Multi-stage Build**, che separa l'ambiente di compilazione da quello di esecuzione.

- **Stage 1 (Build):** Utilizza `node:20-alpine` per installare le dipendenze ed eseguire la build di produzione tramite `ViteG`. L'uso di Alpine Linux garantisce velocità e leggerezza nel processo di CI/CD.
- **Stage 2 (Production):** Il risultato della build viene copiato in un'immagine `nginxG:alpine`. Nginx_G funge da web server statico ad alte prestazioni, configurato appositamente per gestire il routing *client-side* tipico delle Single Page Application (SPA).

4.4 Orchestrazione: Docker Compose_G

Il file `dockerG-compose.yml` coordina l'avvio sincronizzato dei servizi e la gestione delle risorse:

- **Mapping delle porte:** Il frontend è reso accessibile all'host sulla porta 8080, mappata sulla porta interna 80 di Nginx_G.
- **Dipendenze:** È definita una clausola `depends_on` che assicura che il container del backend sia avviato prima di quello del frontend.
- **Sviluppo Agile_G:** Durante la fase di sviluppo, viene utilizzato un *bind mount* per permettere al server Flask_G di riflettere istantaneamente le modifiche al codice senza dover ricostruire l'immagine.

5 Architettura del frontend

Il frontend del sistema è stato sviluppato utilizzando **React_G 19**, adottando il pattern architetturale **MVVM** (Model-View-ViewModel). Questa scelta permette di separare nettamente la logica di business dalla logica di presentazione, facilitando il testing dei componenti e la manutenibilità del codice.

L'organizzazione del progetto segue i principi della *Feature-based Architecture_G* (ispirata a **Bulletproof React_G**), in cui ogni funzionalità del sistema (come l'Editor o lo Storico) è isolata in moduli autocontenuti che includono i propri componenti, hook e store.

5.1 Model

Il Model rappresenta lo stato dei dati e della persistenza. Nel nostro sistema, il Model non è una struttura passiva, ma è gestito tramite store globali realizzati con **Zustand**. Questi store (`useEditorMetaStore`, `useHistoryStore`) fungono da singola fonte di verità (*Single Source of Truth*) per lo stato dell'applicazione. Per garantire la persistenza dei dati senza un database backend, il Model integra il middleware **persist**, che sincronizza automaticamente lo stato dello store con il **LocalStorage** del browser, assicurando che i documenti e lo storico siano disponibili anche dopo il riavvio della sessione.

5.2 View

La View è costituita da componenti React_G dichiarativi e per la maggior parte "dumb" (o *presentational*). Il loro unico compito è il rendering dell'interfaccia utente basandosi sulle *props* ricevute. Le View non contengono logica di business né effettuano chiamate dirette alle API_G; esse delegano ogni interazione dell'utente al ViewModel sottostante tramite callback, garantendo che lo stato di presentazione rimanga snello e facilmente testabile.

5.3 ViewModel

Il ViewModel funge da intermediario tra il Model e la View, ed è implementato attraverso **Custom Hooks** specializzati. Il ViewModel ha la responsabilità di:

- Esporre i dati e lo stato degli store alle View in un formato pronto per la visualizzazione.
- Gestire la logica di interazione complessa (ad esempio, l'orchestrazione delle API_G AI tramite la Facade `aiCall`).
- Manipolare direttamente le istanze di librerie esterne, come **EasyMDE_G** per l'editor Markdown_G, isolandone la complessità tecnica all'interno di hook dedicati.

5.4 Topbar

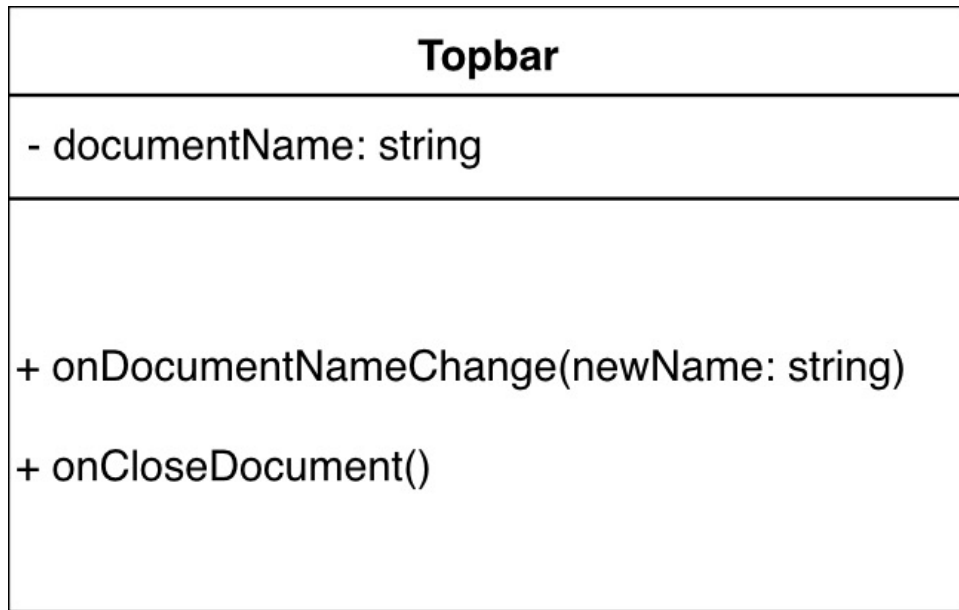


Figura 1: UML TopBar

Il componente Topbar contiene dei pulsanti che consentono di formattare il testo secondo la sintassi Markdown_G.

Attributi:

- `documentName:string` - nome del documento

Metodi:

- `onDocumentNameChange:(newName:string) => void` - callback per aggiornare il nome del documento
- `onCloseDocument:() => void` - callback per chiudere il documento

5.5 Sidebar

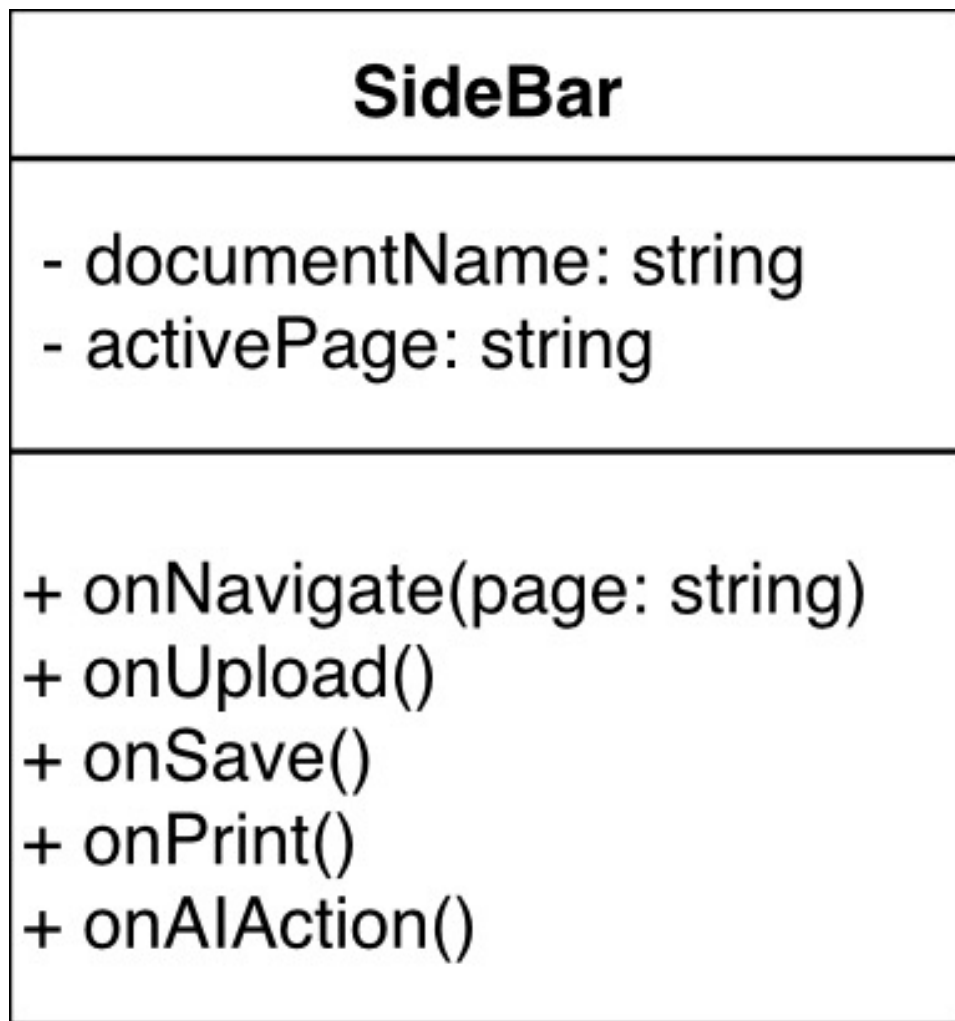


Figura 2: UML SideBar

Il componente Sidebar contiene i pulsanti con le varie funzionalità AI.

Attributi:

- `documentName:string` - nome del documento

Metodi:

- `activePage:'editor' | 'history'` - tipologia di view visualizzata nel corpo principale della pagina
- `onNavigate:(page:'editor' | 'history') => void` - callback per aggiornare la view visualizzata
- `onUpload:() => void` - callback per caricare un documento
- `onSave:() => void` - callback per salvare un documento
- `onPrint:() => void` - callback per stampare un documento
- `onAiAction:() => void` - callback per attivare un'azione AI

5.6 EditorButton

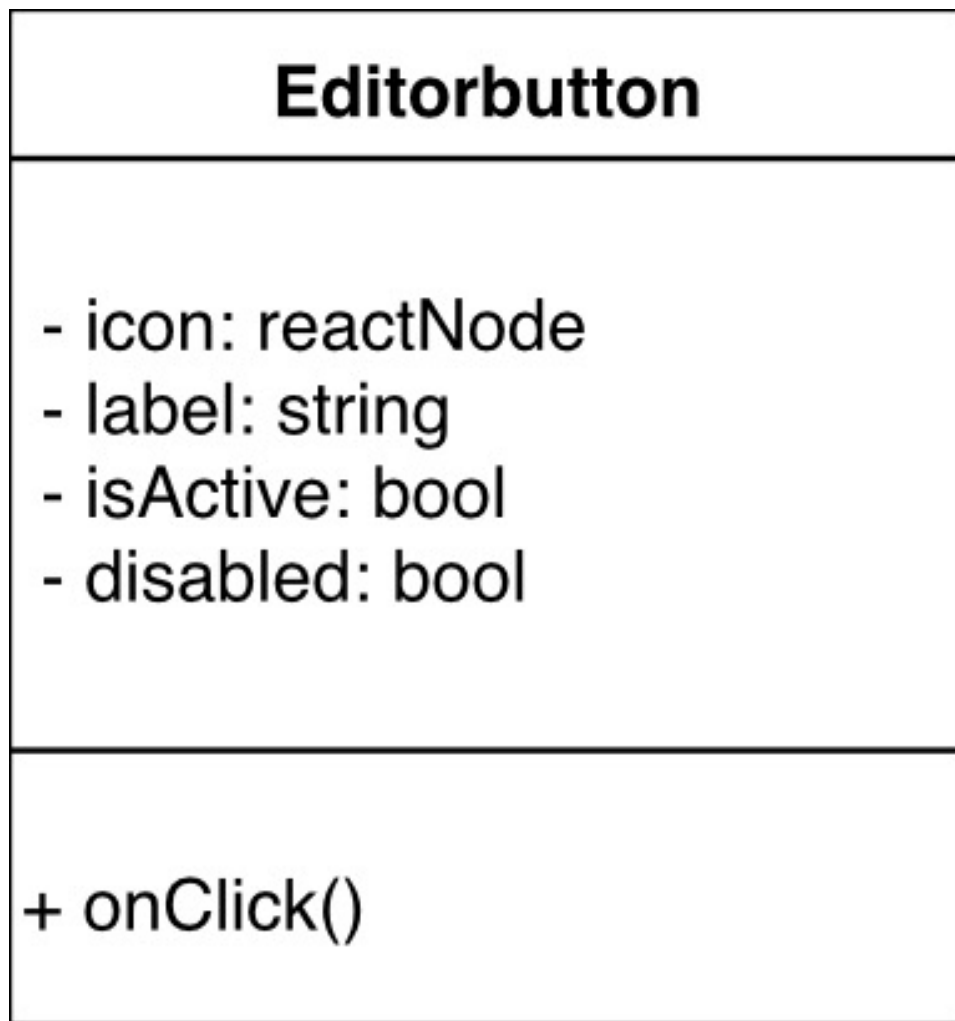


Figura 3: UML EditorButton

Il componente EditorButton è un bottone che viene utilizzato nella sideBar per i bottoni IA.

Attributi:

- `icon:ReactNode` - icona del bottone
- `label:string` - testo del bottone
- `isActive?:boolean` -
- `disabled?:boolean` - true se il bottone non è cliccabile dall'utente

Metodi:

- `onClick:() => void` - callback per gestire il click sul bottone

5.7 SaveDocumentModal

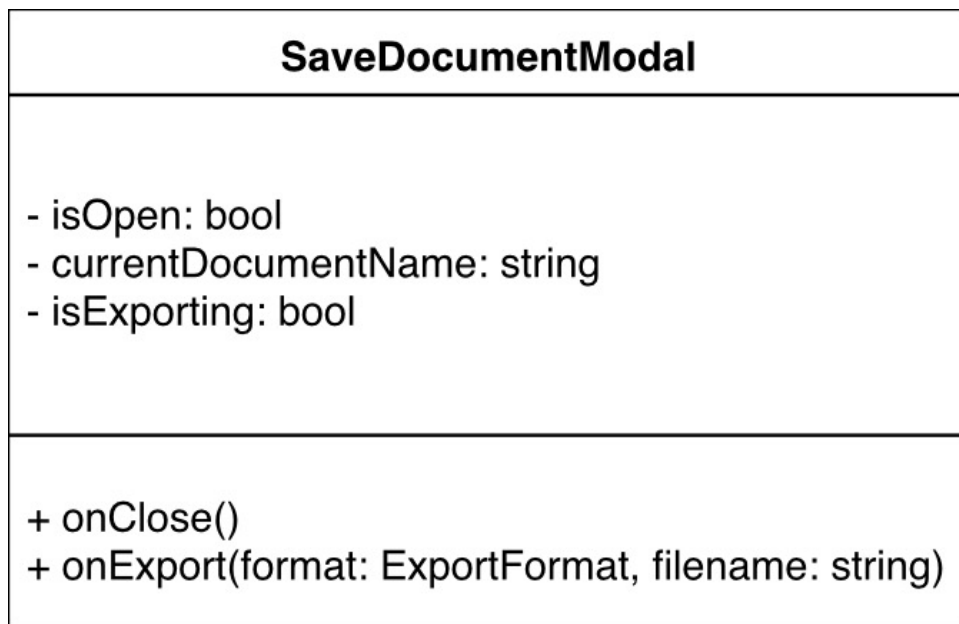


Figura 4: UML SaveDocumentModal

Il componente SaveDocumentModal è il componente che rappresenta il modal per il salvataggio del documento.

Attributi:

- `isOpen:boolean` - true se il modal è aperto
- `currentDocumentName:string` - nome attuale del documento, da mostrare come placeholder nell'input del nome del documento
- `isExporting?:boolean` - true se il documento è in fase di esportazione, per disabilitare i bottoni

Metodi:

- `onClose(): => void` - callback per chiudere il modal
- `onExport:(format:ExportFormat, filename:string) => void` - callback per esportare il documento in un formato specifico

5.8 MarkdownEditor



Figura 5: UML MarkdownEditor

Il componente MarkdownEditor è il componente che rappresenta il corpo principale della pagina e dove viene istanziato EasyMDE_G.

Attributi:

- `textAreaRef:ReactG.RefObject<HTMLTextAreaElement | null>` - riferimento alla textarea su cui istanziare EasyMDE_G

Metodi:

- `onFileDrop?:(file:File) => void` - callback per gestire il drop di un file sulla textarea

5.9 HistoryList

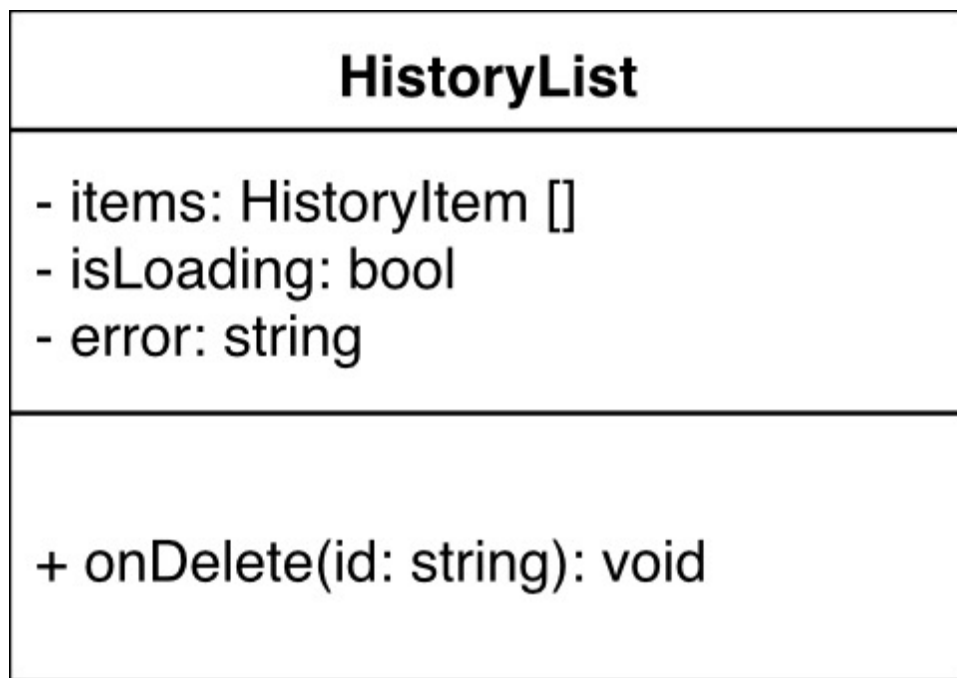


Figura 6: UML HistoryList

Il componente HistoryList è il componente che rappresenta la lista delle interazioni con l'IA.

Attributi:

- `items:HistoryItem[ ]` - lista delle interazioni con l'IA da mostrare nella cronologia
- `isLoading:boolean` - true se la cronologia è in fase di caricamento, per mostrare un indicatore di caricamento
- `error:string | null` - messaggio di errore da mostrare se c'è stato un errore nel caricamento della cronologia

Metodi:

- `onDelete(id:string) => void` - callback per gestire la cancellazione di una interazione dalla cronologia

5.10 HistoryCard

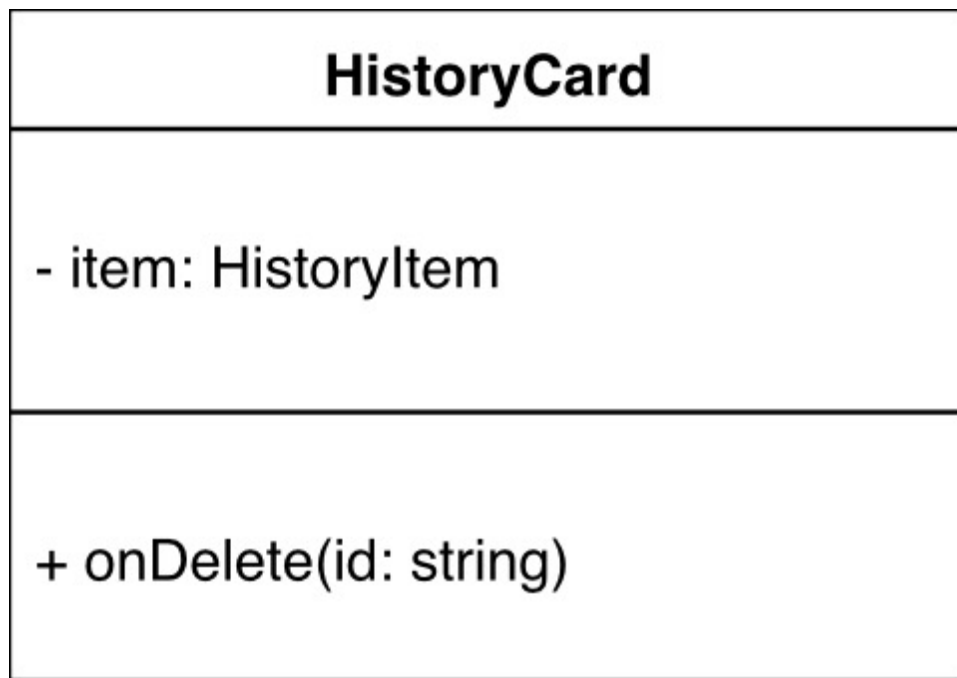


Figura 7: UML HistoryCard

Il componente HistoryCard è il componente che rappresenta la scheda di una interazione con l'IA.

Attributi:

- `item:HistoryItem`

Metodi:

- `onDelete:(id:string) => void` - callback per gestire la cancellazione di una interazione dalla cronologia

5.11 UseEditor

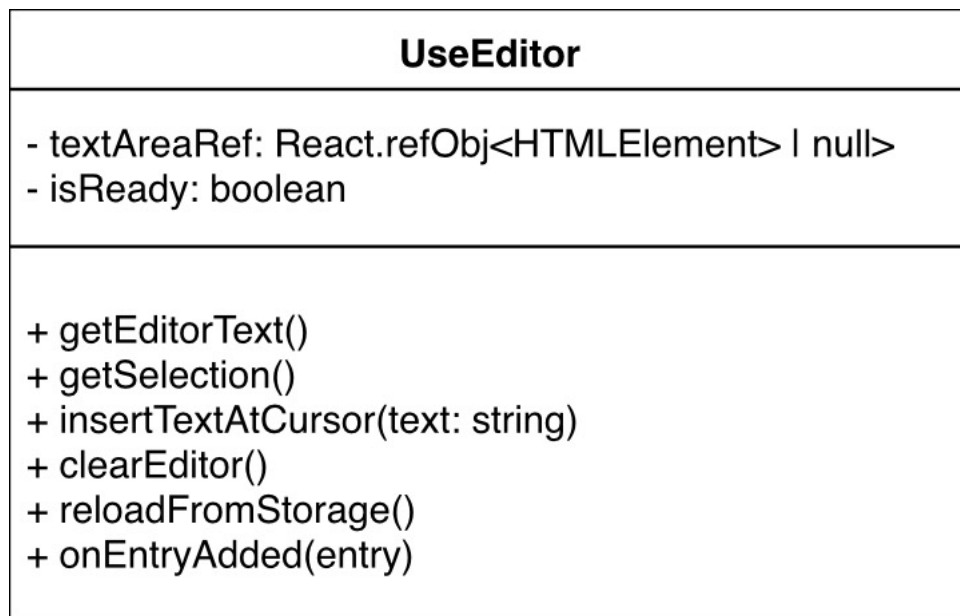


Figura 8: UML UseEditor

Hook per l'editor

Attributi:

- `textArearef: React.RefObject<HTMLTextAreaElement | null>` - riferimento alla textarea su cui istanziare `EasyMDEG`
- `isReady: boolean` - true quando l'editor è pronto all'utilizzo

Metodi:

- `getEditorText: () => string` - restituisce il testo presente nell'editor
- `getSelection: () => string` - restituisce il testo attualmente selezionato nell'editor
- `insertTextAtCursor: (text: string) => void` - inserisce il testo nella posizione del cursore
- `clearEditor: () => void` - cancella il contenuto dell'editor
- `reloadFromStorage: () => void` - ricarica il contenuto dell'editor
- `onEntryAdded?: (entry: unknown) => void` - callback per gestire l'aggiunta di una nuova voce all'editor

5.12 Rappresentazione UML per il frontend

6 Design Pattern

6.1 Strategy pattern

Il design pattern comportamentale Strategy definisce una famiglia di algoritmi, li incapsula in classi separate e li rende intercambiabili in modo dinamico.

6.1.1 Problema

L'architettura iniziale del backend presentava un'eccessiva complessità logica dovuta alla gestione di molteplici modelli LLM e diverse logiche di prompting. La presenza di numerosi blocchi condizionali (if/else o switch) all'interno dell'entry point (`app.py`) rendeva il codice rigido, difficile da scalare e violava il principio di singola responsabilità (Single Responsibility Principle).

6.1.2 Applicazione: `promptG strategies (ai_strategies.py)`

Questa prima applicazione del pattern gestisce la trasformazione del testo grezzo fornito dall'utente in un input strutturato e ottimizzato per l'IA.

BaseAIStrategy Definisce il contratto comune per tutte le strategie di generazione dei `promptG`.
Proprietà e Metodi:

- **temperature:** Definisce il livello di creatività dell'output (es. valori più alti per operazioni creative, più bassi per task analitici).
- **build(text: str):** Impone a ogni sottoclasse di restituire una tupla contenente due stringhe: il System `promptG` (il ruolo dell'IA) e lo User `promptG` (il task specifico).

SimplePromptStrategy Utilizzata per le operazioni editoriali standard (riassunto, correzione grammaticale, riscrittura, distant writing_G). Costruisce `promptG` diretti impostando un ruolo e istruzioni ferree per evitare preamboli discorsivi nella risposta dell'IA.

Attributi:

- **role (str):** Definisce l'identità dell'assistente (es. "Sei un correttore di bozze").
- **task (str):** Definisce l'istruzione specifica da eseguire.

TranslationStrategy Sottoclasse concreta specializzata nella gestione delle attività di traduzione multilingua. Questa classe ottimizza il `promptG` per garantire che l'LLM operi esclusivamente come traduttore professionale, mantenendo la fedeltà linguistica.

Attributi:

- **language(str):** Stringa che definisce la lingua di destinazione (es. "inglese", "cinese mandarino"). Questo attributo viene utilizzato per parametrizzare dinamicamente la richiesta al modello.

Metodi:

- **build(text):** Implementa la logica di costruzione del `promptG` impostando un System `PromptGG` rigido che definisce il ruolo (traduttore professionista) e i vincoli di output (nessuna frase introduttiva). Lo User `PromptGG` include l'istruzione di traduzione specifica verso la lingua memorizzata nell'attributo `language`.

DeBonoHatStrategy Sottoclasse concreta progettata per implementare il metodo dei "Sei Cappelli per Pensare" di Edward de Bono. Questa strategia guida l'LLM verso un'analisi laterale e strutturata, forzando il modello a osservare il testo da una singola prospettiva cognitiva alla volta (emotiva, logica, creativa, ecc.).

Attributi:

- `hat_name(str)`: Memorizza il nome o il colore del cappello. Serve a definire l'identità dell'IA.
- `focus(str)`: Memorizza l'area di analisi specifica. Serve a guidare il focus del modello durante la generazione.

Metodi:

- `build(self, text)`: Utilizza le f-string per iniettare i valori di `hat_name` e `focus` all'interno dei `promptG`.

6.1.3 Applicazione: model strategies (model_strategies.py)

Questa seconda applicazione del pattern isola completamente i dettagli tecnici (`EndpointG`, `APIG` Keys, payload JSON) necessari per comunicare con i diversi provider di Intelligenza Artificiale.

AIModelStrategy interfaccia comune per tutti i modelli linguistici IA.

Metodi:

- `generate(self, system_prompt, user_prompt)`: Definisce il contratto per l'invocazione del modello. Ogni classe concreta deve implementare la logica specifica per inviare i `promptG` al fornitore e restituire la risposta come stringa pulita.

Implementazioni Concrete (ZucchettiLlamaStrategy e Gemma3Strategy) Il sistema prevede due implementazioni concrete che espongono in modo polimorfo i modelli Llama 3.2_G (per operazioni standard e traduzioni) e Gemma 3_G (per il pensiero laterale dei Sei Cappelli). Entrambe le classi implementano la logica di chiamata `APIG` (tramite modulo OpenAI compatibile), formattano la richiesta ed estraggono esclusivamente il contenuto testuale della risposta, isolandolo dai metadati di rete.

Metodi:

- `generate(system_prompt, user_prompt)`: Implementa la logica di chiamata `APIG` tramite il modulo OpenAI, gestisce la struttura della richiesta inviando i due messaggi (System e User) nel formato richiesto dal protocollo ChatCompletion ed estrae e restituisce esclusivamente il contenuto testuale della risposta, isolandolo dai metadati della chiamata (come i token consumati o gli ID di richiesta).

6.1.4 Caching (get_model)

La funzione `get_model(model_class)` gestisce un dizionario globale privato (`_model_instances`) per la memorizzazione delle istanze, che funge quindi da cache. Quando il controller richiede una strategia di modello, la funzione verifica_G se ne esiste già un'istanza nel dizionario: se non esiste, la crea e la salva; alle chiamate successive per la stessa classe, restituisce il riferimento già memorizzato. Questa soluzione architetturale ottimizza l'uso delle risorse prevenendo l'apertura di connessioni multiple non necessarie ai client `APIG`.

6.2 Registry pattern

Il pattern Registry funge da catalogo centralizzato e configuratore delle operazioni disponibili, disaccoppiando totalmente il Controller (`app.py`) dalla logica di creazione delle dipendenze.

6.2.1 Problema

L'integrazione tra le diverse strategie di prompting e i diversi LLM poneva un problema di accoppiamento rigido e di complessità gestionale. Senza un punto di coordinamento centrale, il Controller avrebbe dovuto:

- Conoscere l'esistenza di tutte le classi concrete e importarle direttamente.
- Gestire manualmente la logica di accoppiamento tra `promptG` e modelli.
- Contenere lunghi blocchi condizionali per istanziare la strategia corretta in base alla stringa ricevuta dal frontend.

Questa struttura avrebbe violato il principio di Inversione delle Dipendenze, rendendo l'aggiunta di una nuova operazione un processo rischioso e richiedendo la modifica del codice di routing principale ogni volta che viene introdotta una nuova funzionalità.

6.2.2 Applicazione: `operation_registry.py`

Il modulo funge da punto di giunzione dell'intera architettura, orchestrando il legame tra le strategie di contenuto (`promptG`) e le strategie di esecuzione (modelli).

OperationConfig La struttura `OperationConfig` è definita tramite il decoratore `@dataclass` ed è progettata per garantire un accoppiamento debole tra i componenti. Agisce come un contenitore che incapsula le dipendenze necessarie a ogni singola operazione.

Attributi:

- `model_class` (`Type[AIModelStrategy]`): è il riferimento al tipo della classe del modello da istanziare, la cui creazione effettiva è delegata al sistema di caching (`get_model`) visto in precedenza.
- `promptG_strategy` (`BaseAIStrategy`): è un'istanza preconfigurata della strategia di `promptG` specifica per quell'operazione.

OPERATION_REGISTRY Il nucleo del pattern è implementato come un dizionario globale che associa identificatori testuali univoci (es. `"summary"`, `"green_hat"`) a oggetti `OperationConfig`. Questa architettura garantisce tre proprietà fondamentali:

- **Adesione al principio Open-Closed:** Per aggiungere una nuova funzionalità è sufficiente registrare una nuova riga nel dizionario di configurazione. Il codice del controller rimane del tutto inalterato, chiuso alle modifiche ma aperto alle estensioni.
- **Routing dinamico dei modelli:** Il registro permette di assegnare il modello LLM più adatto a una specifica operazione. Ad esempio, nel sistema le operazioni editoriali e di traduzione sono instradate nativamente su Llama 3.2_G, mentre le logiche di pensiero laterale dei Sei Cappelli sono delegate a Gemma 3:4B_G.
- **Robustezza e fallback:** In caso di operazione non riconosciuta o assente nel payload della richiesta, il sistema fornisce una configurazione di default in modo trasparente tramite la costante `DEFAULT_OPERATION = "summary"`, sfruttando il metodo `.get()` del dizionario ed evitando crash dell'applicazione.

7 Tracciamento dei requisiti

In questa sezione è stato riportato il tracciamento dei requisiti del progetto "Second Brain" con lo scopo di avere una panoramica completa sui requisiti stabiliti nel documento dell'Analisi dei Requisiti_G v.2.0.0 e sul loro conseguimento.

7.1 Notazione dei requisiti

Il codice identificativo segue il formato:

$$R[Importanza] - [Tipo] - [Numero]$$

Dove le voci assumono i seguenti significati:

Importanza

Indica il grado di priorità del requisito:

- **O (Obbligatorio):** Requisito esplicitamente richiesto dall'azienda Zucchetti. La sua mancata implementazione implica il fallimento degli obiettivi primari del progetto.
- **D (Desiderabile):** Requisito non strettamente vincolante ma che il gruppo si impegna ad implementare per migliorare il prodotto.
- **OP (Opzionale):** Requisito di priorità minore, la cui implementazione può essere rimandata.

Tipo

Indica la natura del requisito:

- **F (Funzionale):** Descrive le funzionalità e i servizi che il sistema deve fornire.
- **V (Vincolo):** Descrive limitazioni tecnologiche, normative o implementative imposte esternamente.
- **Q (Qualità):** Descrive le caratteristiche qualitative del sistema (es. performance, sicurezza, manutenibilità).

Numero

Un numero intero progressivo univoco per tipologia di requisito.

7.2 Tracciamento dei requisiti funzionali

Codice	Descrizione	Stato
RO-F-1	Il sistema deve fornire un'area di testo per l'inserimento e la modifica del testo con supporto alla sintassi Markdown _G standard (CommonMark).	Conseguito
RO-F-2	Il sistema deve informare l'utente con un messaggio in caso di perdita della connessione Internet durante l'utilizzo dell'editor.	...
RO-F-3	Il sistema deve informare l'utente con un messaggio se l'editor non risponde ai comandi.	...
RO-F-4	L'editor deve compilare la sintassi Markdown _G in tempo reale.	...
RO-F-5	Deve essere presente un'area di visualizzazione del documento formattato rispetto alla sintassi Markdown _G .	...
RO-F-6	Il sistema deve informare l'utente con un messaggio se il pannello di anteprima non si aggiorna correttamente.	...

Codice	Descrizione	Stato
RO-F-7	Il sistema deve permettere all'utente di selezionare porzioni di testo tramite mouse o tastiera.	...
RO-F-8	Il sistema deve permettere all'utente di modificare porzioni di testo selezionate.	...
RO-F-9	Il sistema deve permettere all'utente di copiare una parte di testo selezionato.	...
RO-F-10	Il sistema deve permettere all'utente di incollare un testo nell'area di testo.	...
RO-F-11	Il sistema deve permettere all'utente di tagliare il testo selezionato.	...
RO-F-12	Il sistema deve permettere all'utente di annullare l'ultima modifica effettuata.	...
RO-F-13	Se non sono presenti operazioni annullabili, il sistema non esegue alcuna azione al comando Undo.	...
RO-F-14	Il sistema deve permettere all'utente di ripristinare le modifiche precedentemente annullate.	...
RO-F-15	Se non sono presenti operazioni ripristinabili, il sistema non esegue alcuna azione al comando Redo.	...
RO-F-16	Nell'editor deve essere presente una barra degli strumenti (toolbar _G) per applicare formattazioni Markdown _G senza scrivere manualmente la sintassi.	...
RO-F-17	Il sistema deve permettere l'applicazione del grassetto al testo tramite toolbar _G .	...
RO-F-18	Il sistema deve permettere l'applicazione del corsivo al testo tramite toolbar _G .	...
RO-F-19	Il sistema deve permettere l'applicazione della sottolineatura al testo tramite toolbar _G .	...
RO-F-20	Il sistema deve permettere di creare intestazioni tramite toolbar _G .	...
RO-F-21	Il sistema deve permettere l'inserimento di elementi di struttura (tabelle, separatori orizzontali) tramite toolbar _G .	...
RO-F-22	Il sistema deve permettere l'inserimento di liste numerate tramite toolbar _G .	...
RO-F-23	Il sistema deve permettere l'inserimento di elenchi puntati tramite toolbar _G .	...
RO-F-24	Il sistema deve permettere l'inserimento di link e riferimenti tramite toolbar _G .	...
RO-F-25	L'utente può applicare più stili contemporaneamente (es. grassetto, corsivo, sottolineatura) alla stessa sezione di testo.	...
RO-F-26	L'utente può annullare una formattazione di una porzione di testo selezionato riapplicandola dalla toolbar _G .	...
RO-F-27	L'utente può estendere la formattazione a una porzione di testo parzialmente formattato riapplicandola dalla toolbar _G .	...
RO-F-28	L'utente può creare sottoliste attivando la creazione di una lista dalla toolbar _G quando il cursore è già all'interno di una lista.	...
RO-F-29	L'utente può scegliere una combinazione di formattazioni dalla toolbar _G quando non è presente testo selezionato, per applicarle al testo che scriverà successivamente.	...
RO-F-30	Il sistema deve permettere all'utente di selezionare porzioni di testo da inviare all'agente esterno.	...
RO-F-31	Il sistema permette di inviare del testo ad un agente esterno.	...

Codice	Descrizione	Stato
RO-F-32	L'utente deve poter accedere alle azioni sul testo tramite agente esterno in ogni momento.	...
RO-F-33	Il testo generato viene prima mostrato come proposta, separata visivamente dal testo dell'utente.	...
RO-F-34	Il testo in stato di proposta include le opzioni di conferma, annullamento e rigenerazione.	...
RO-F-35	Il sistema non permette la modifica del testo generato mentre è in stato di proposta.	...
RO-F-36	Il testo generato dall'agente esterno, una volta confermato, viene inserito nella posizione del cursore se presente, altrimenti in fondo al testo.	...
RO-F-37	Una proposta che viene confermata viene inserita nel documento nell'editor e la sezione di proposta viene rimossa.	...
RO-F-38	Una volta generato e confermato, il testo dell'agente esterno è modificabile tramite le stesse funzionalità dell'editor disponibili per il testo utente.	...
RO-F-39	Una proposta annullata comporta la rimozione del testo generato dalla sezione di proposta e la rimozione della sezione stessa.	...
RO-F-40	Una proposta che viene rigenerata viene rimandata all'agente esterno con gli stessi parametri e la nuova risposta sovrascrive la proposta corrente.	...
RO-F-41	Il sistema deve informare l'utente con un messaggio in caso di errore durante la comunicazione con il servizio LLM.	...
RO-F-42	L'utente può annullare un'operazione AI in corso prima del suo completamento.	...
RO-F-43	In seguito all'annullamento di un'operazione AI, il documento rimane nello stato precedente alla richiesta.	...
RO-F-44	Se la generazione di testo si blocca prematuramente, rimane ciò che è già stato generato, in attesa che l'utente confermi, rigeneri o elimini la proposta.	...
RO-F-45	L'agente esterno applica modifiche esclusivamente al testo selezionato dall'utente. Un contesto aggiuntivo può essere fornito solo a scopo interpretativo e non deve essere alterato.	...
RO-F-46	L'utente può consultare il registro delle generazioni AI effettuate.	...
RO-F-47	Il sistema deve gestire il caso in cui lo storico delle generazioni AI sia vuoto, informando l'utente con un messaggio appropriato.	...
RO-F-48	Ogni generazione di testo dell'agente esterno viene inclusa nel registro delle generazioni, incluse quelle annullate dall'utente.	...
RO-F-49	Il sistema deve permettere all'utente di visualizzare il testo completo di una generazione dallo storico AI.	...
RO-F-50	L'utente deve poter riassumere tutto o una parte selezionata del testo usando un agente esterno.	...
RO-F-51	L'utente deve poter usare un agente esterno per riscrivere tutto o una parte selezionata del testo migliorandolo seguendo determinate specifiche come sintassi, stile e chiarezza.	...
RO-F-52	L'utente deve poter tradurre il testo usando un agente esterno in: inglese, italiano, tedesco, francese, spagnolo e cinese mandarino.	...
RO-F-53	Se l'utente avvia una traduzione senza selezionare la lingua di destinazione, il sistema lo notifica e blocca l'operazione.	...

Codice	Descrizione	Stato
RO-F-54	Il sistema permette di avviare la correzione grammaticale del testo tramite agente esterno.	...
RO-F-55	Se non vengono rilevati errori grammaticali, il sistema informa l'utente dell'esito positivo della correzione.	...
RO-F-56	Il sistema consente all'utente di avviare l'analisi critica del testo selezionato o dell'intero documento utilizzando il cappello di De Bono scelto.	...
RO-F-57	Se il testo è insufficiente per l'analisi critica, il sistema notifica l'utente e interrompe l'operazione.	...
RO-F-58	Se l'utente avvia l'analisi critica senza selezionare un cappello di De Bono, il sistema lo notifica e blocca l'operazione.	...
RO-F-59	Il sistema permette l'invio di un prompt _G scritto dall'utente per la generazione di testo tramite agente esterno.	...
RO-F-60	Il sistema permette la modifica del prompt _G da inviare all'agente esterno prima dell'invio.	...
RO-F-61	L'agente esterno può generare testo basandosi su un prompt _G fornitogli dall'utente.	...
RO-F-62	Se il prompt _G fornito dall'utente è vuoto o incompleto, il sistema notifica l'utente e impedisce l'invio.	...
RO-F-63	L'utente può caricare un file dal proprio dispositivo all'interno dell'editor.	...
RO-F-64	Il sistema deve supportare il caricamento di file in formato Markdown _G (.md, .markdown _G) e testo semplice (.txt).	...
RO-F-65	Il sistema deve supportare il caricamento tramite dialogo di esplorazione file (file picker).	...
RO-F-66	Prima di procedere con il caricamento di un file, il sistema chiede conferma all'utente.	...
RO-F-67	Il sistema deve informare l'utente con un messaggio di esito (successo o errore) al termine di ogni tentativo di caricamento file.	...
RO-F-68	Il sistema deve validare il formato del file selezionato e impedire il caricamento di formati non supportati.	...
RO-F-69	Il sistema deve validare le dimensioni del file selezionato e impedire il caricamento di file che superano la dimensione massima consentita.	...
RO-F-70	Il sistema deve impedire il caricamento di file corrotti o illeggibili.	...
RO-F-71	Il sistema deve permettere all'utente di rimuovere il file che ha caricato.	...
RO-F-72	L'utente può annullare l'operazione di rimozione del file caricato, lasciando l'editor nel suo stato precedente.	...
RO-F-73	Il sistema deve essere in grado di convertire il documento da Markdown _G al formato selezionato dall'utente.	...
RO-F-74	Il sistema deve permettere di scaricare il file nel dispositivo dell'utente.	...
RO-F-75	L'operazione di esportazione o stampa non deve alterare né chiudere il documento originale nell'editor.	...
RO-F-76	Il sistema deve informare l'utente con un messaggio di esito al termine di ogni operazione di esportazione.	...

Codice	Descrizione	Stato
RO-F-77	Il sistema deve consentire l'esportazione del documento in formato Markdown _G (.md).	...
RO-F-78	Il sistema deve controllare che il range di pagine inserito dall'utente sia valido (non superiore al numero di pagine del documento e non negativo).	...
RO-F-79	Il sistema deve permettere di collegare e inviare il documento a un dispositivo di stampa.	...
RO-F-80	Il sistema deve informare l'utente con un messaggio in caso di errore durante l'invio del documento alla stampante.	...

Tabella 11: Tracciamento dei requisiti funzionali_G

7.3 Tracciamento dei requisiti di qualità

Codice	Descrizione	Stato
RO-Q-1	Il tempo di risposta dell'interfaccia utente (aggiornamento anteprima, applicazione formattazioni) deve essere inferiore a 500 ms nel 95% dei casi, misurato su una sessione utente attiva di almeno 30 minuti.	...
RO-Q-2	Il tempo medio di generazione di una risposta da parte del servizio LLM non deve superare i 10 secondi, misurato su un campione di almeno 20 richieste consecutive in condizioni di utilizzo normale.	...
RO-Q-3	Il tasso di errori nelle operazioni principali (salvataggio, apertura file, chiamate AI) non deve superare l'1%, misurato sul totale delle operazioni eseguite in una sessione utente attiva di almeno 30 minuti.	...
RO-Q-4	La percentuale di codice coperta dai test automatici (test coverage) deve essere almeno il 70%, misurata sull'insieme completo dei moduli rilasciati ad ogni versione.	...
RO-Q-5	La complessità ciclomatica (McCabe) di ogni funzione/metodo del codice prodotto non deve superare il valore 8.	...
RO-Q-6	La documentazione prodotta deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	...
RO-Q-7	Il sistema deve essere sviluppato rispettando le Norme di Progetto del gruppo ByteMe.	...
RO-Q-8	Il sistema deve essere verificato secondo il Piano di Qualifica prima di ogni rilascio.	...
RO-Q-9	Il manuale utente _G deve documentare tutte le funzionalità obbligatorie del sistema e deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	...
RO-Q-10	Il manuale di installazione e deployment del sistema deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	...

Tabella 12: Tracciamento dei requisiti di qualità_G

7.4 Tracciamento dei requisiti di vincolo

Codice	Descrizione	Stato
RO-V-1	L'applicazione deve essere accessibile tramite browser web. I browser supportati sono: Google Chrome ($\geq$ v110), Mozilla Firefox ($\geq$ v110), Microsoft Edge ($\geq$ v110), Safari ($\geq$ v16).	...
RO-V-2	Il sistema deve utilizzare la codifica Unicode (UTF-8) per la rappresentazione interna e lo storage del testo.	...
RO-V-3	Il formato di input e di storage del testo deve essere Markdown _G , in conformità allo standard CommonMark.	...
RO-V-4	Il sistema deve integrarsi con un servizio LLM tramite API _G REST compatibili con lo standard OpenAI.	...

Tabella 13: Tracciamento dei requisiti di vincolo_G